

FinRep — Technical Architecture & System Breakdown

Engineering reference · monorepo, engine, API, web, data model, infra

Repo: [@finrep/monorepo](#) v0.1.0 · **Updated:** June 2026 · **Audience:** the engineering team (you)

1. Executive technical summary

FinRep is a multi-tenant SaaS that converts uploaded school **trial balances** (Excel/CSV) into a reproducible **ReportBundle** of nonprofit financial statements (SOA, SFP, SCF, Net Assets). The system is a **pnpm + Turborepo monorepo** with a hard separation between a **pure, deterministic calculation core** and all I/O/UI around it.

The defining architectural decisions:

- **Purity boundary.** [@finrep/engine](#) is zero-I/O, zero-UI, clock-free. Same inputs → byte-identical outputs, enforced by a regression + purity test suite. This is the crown jewel and the lowest-risk part of the system.
 - **Reproducibility contract.** A `StatementSnapshot` is a *derived* artifact, fully reconstructable from `(immutable import.rows) + (mapping@version) + (chart@version) + (engineVersion)`. Imports are append-only; mappings/charts are versioned, never mutated in place.
 - **Tenant isolation contract.** `Organization 1-< School (THE TENANT) 1-< every domain table (school_id)`. Users are *global* identities; access is granted via `memberships(user_id, school_id, role)` and enforced by a `RolesGuard`.
 - **ESM/dist consumption constraint.** The web app aliases engine *source* via Vite (esbuild transpiles `.ts` on the fly); the API cannot, so it consumes the engine's **built dist/**. Turbo's `^build` ordering guarantees deps build first.
-

2. Technology stack

Layer	Technology	Notes
Monorepo	pnpm 11 workspaces + Turborepo 2.5	<code>apps/*</code> , <code>packages/*</code> ; <code>dependsOn: ["^build"]</code> ordering
Language	TypeScript 5.6 (ESM throughout)	<code>type: module</code> ; NodeNext (api) vs Bundler (engine/ingestion)
Engine/Ingestion build	tsup	Emits <code>dist/</code> consumed by the api
API	NestJS 11 (Express)	ESM, <code>moduleResolution: NodeNext</code>
Auth	<code>@nestjs/jwt</code> + Node <code>crypto</code> (PBKDF2)	Access/refresh JWTs, rotation, logout
ORM / DB	Prisma + PostgreSQL 16	UUID PKs, JSONB snapshots, Decimal(18,2) money
Web	React 19 + Vite 7 + Tailwind	<code>react-router-dom</code> , <code>framer-motion</code> , axios
Spreadsheets	SheetJS (<code>xlsx</code>) in, ExcelJS out	ExcelJS lazy-loaded (~500 KB) only on export
Container	Docker multi-stage + <code>turbo prune --docker</code>	<code>node:22-slim</code> , pinned Prisma engine target
Tests	Vitest (engine/ingestion)	regression, purity, validation, opening-NA suites

3. Monorepo topology

```
@finrep/monorepo
├── apps/
│   ├── api/      @finrep/api    NestJS backend (ESM, consumes engine dist)
│   └── web/      @finrep/web    React 19 SPA (aliases engine SOURCE via Vite)
├── packages/
│   ├── engine/   @finrep/engine  pure financial calc core (zero I/O)
│   ├── ingestion/ @finrep/ingestion file bytes → NormalizedRow[]
│   └── db/        @finrep/db     Prisma schema + generated client
├── scripts/      verify-engine.mjs, gen-sample-data.mjs, derive-opening.mjs
├── sample-data/  TB_*.xlsx fixtures
├── docker-compose.yml  postgres + api (+ web profile)
└── turbo.json     build/dev/lint/test/typecheck task graph
```

3.1 The engine-consumption constraint (read this before touching builds)

The engine/ingestion sources use `moduleResolution: "Bundler"` with `.js`-suffixed relative specifiers and export `./dist/index.js`. Plain Node / `tsc` (NodeNext) **cannot** resolve `./types/rows.js` against a `.ts` file. Therefore:

- **web** → Vite `resolve.alias` maps `@finrep/engine` and `@finrep/ingestion` to `packages/*/src/index.ts`; esbuild transpiles on the fly → instant HMR, no prebuild.
- **api** → consumes the package `main / exports` → `./dist/index.js`. **No tsconfig path alias to source.** Workspace deps must be built first; `turbo build --filter=@finrep/api` builds `@finrep/db` (Prisma client) + `@finrep/engine` before the api.

This asymmetry is intentional and load-bearing. Don't "simplify" it by pointing the api at source — Node's resolver will reject the `.js`-suffixed imports against `.ts` files.

4. @finrep/engine — the calculation core

Pure TypeScript. **ZERO UI, ZERO I/O**. Inputs as args → outputs as return values. Never reads the clock (caller may pass `generatedAt` at the I/O boundary).

4.1 Data contract

```
interface NormalizedRow { acct: number; desc: string; total: number }
// total is a SIGNED amount: debit positive, credit negative.
type Dataset = NormalizedRow[]
```

This single signed-amount convention is the source of the negation pattern throughout: revenue accounts carry natural credit (negative) balances and are negated to display positive.

4.2 Standard Chart of Accounts (SCoA) layer

A versioned bundle decoupling *account numbers* from *statement categories*:

- `categories.ts` — `SCOA_CATEGORIES`: category → { `section`, `sign`, `rollupLine`, `includedInTotals` }. Revenue carries `sign:-1` (encodes the legacy `-(sum)` as metadata, used for lineage — *not* a separate code path). Behavioral quirks preserved as data: `ancillary` (910/911/918) is mapped but `includedInTotals:false`; `studActExp` has no mapped accounts (sum 0).
- `chart.ts` — `StandardChart` = { `standardChartVersion`, `categories`, `mapping` }; pure helpers `categoryOf`, `sumByAccts` (legacy `sumA`), `sumByCategory` (legacy `sumC`), plus `rowsBy*` variants that capture source rows for lineage. Lookup semantics (numeric-key, exact reduce order) match the legacy engine exactly.
- `defaultMapping.ts` — `ACCT_MAP` / `DEFAULT_MAPPING` (account → category).

4.3 Calculation pipeline

`generateReports(args)` → `ReportBundle` orchestrates, running each calculator for whatever datasets are present (CY required; PY/audit optional):

```
calcSOA → Statement of Activities (revenue/expense rollups, netChange)
calcSFP → Statement of Financial Position (uses SOA-derived netAssets end)
calcSCF → Statement of Cash Flows (needs a "beginning" set: audit > py)
calcNetAssets → roll-forward column (begin + netChange = end)
validateDataset + findUnmapped → data-quality surface
buildXxxLineage → parallel traceability tree (numbers unchanged)
```

Net-asset roll-forward is the spine: `cyNAEnd = school.netAssetsBegin + cy.netChange`, and SFP consumes `naEnd` (so the balance sheet ties to the activity statement by construction).

Domain logic faithfully ported (verbatim) from a legacy single-file engine — notable preserved behaviors:

- **SOA tuition** uses an explicit acct list `[401..405,409]` via `sumByAccts`, **not** the category sum, to match legacy.
- **SFP acct-120 reclass** by description: *Suspense / Payment at Institution* → cash; *Prepaid* → prepaid; remainder → tuition receivable. **acct-200 lease split**: `lease` desc → `leaseCurr`, remainder → `apAccrued`, with `Math.abs`.
- **SCF** working-capital deltas vs a beginning dataset (`audit` preferred, else `py`); AR = all acct-120 incl. TMS suspense; PP&E purchases net of ROU reclass (160→150); depreciation from acct 865.

4.4 Opening net assets recovery (`openingNetAssets.ts`)

Management TBs *omit* the opening-equity row, so the figure a user would type into "Net assets — beginning" is recoverable as the TB's own imbalance:

```
sum(present rows) = -openingEquity = openingNetAssets    (a credit, shown +)
```

Three cases by confidence: `equity-row` (300-series present → read directly; complete TB nets to zero), `plug` (balance-sheet accounts present, equity omitted → imbalance is opening NA, but *should be confirmed* — the plug absorbs any other omission too), `unavailable` (pure rev/exp extract → imbalance is period activity, not opening; fall back to manual/roll-forward).

4.5 Validation (`validate.ts`)

debits=credits is **data-model aware**: if an equity/opening row (300–399) is present it's a complete TB and *must* net to zero (nonzero = real UNBALANCED error); if equity is omitted (management-TB case) the strict check is N/A → `balanced:true` + an informational issue. `findUnmapped` flags `acct >= 400` rows with nonzero balance and no chart mapping (this is what surfaces e.g. account 462 in "Accounts Requiring Review"); `ancillary`-mapped accts are never flagged.

4.6 Lineage & versioning

Every statement builds a parallel `ReportLineage` mapping each line → `{ scoaCategory, statement, sign, value, sources[] }` — the traceability backbone for audit. `version.ts` pins `ENGINE_VERSION=0.1.0`, `MAPPING_VERSION=map-v1`, `STANDARD_CHART_VERSION=scoa-v1`; these stamp `ReportMeta` and are the keys of the reproducibility contract.

5. `@finrep/ingestion` — the only place bytes become rows

The ONLY place file bytes are turned into normalized rows. Pluggable adapters.

- **Adapters** — `excelAdapter` (SheetJS) and `csvAdapter`, behind a `registry` (`ingest`, `getAdapter`). Each yields `IngestionResult { rows: NormalizedRow[], metadata: SheetMetadata }`.
- **Metadata** (`metadata.ts`, pure) — `extractSheetMetadata`, `detectFiscalYear`, `detectExplicitDate`, `detectAuditStatus`.
- **Classification** (`classify.ts`, pure, byte-free function of `{ fileName, metadata }`) — `classifyRole` → `{ role: cy|py|audit|ignore|unknown, confidence 0..1, signals }`; `resolveRoles` fills cy/py/audit slots

and **surfaces conflicts rather than silently misassigning**; `inferPeriod` derives date + type. Calibration note baked in: prior vs audited can't be split by fiscal year alone (both FY25 in the sample set), so the role **keyword** is weighted above FY; FY is only a CY-vs-PY tiebreaker. Lives here (not in web) because it's a pure, vitest-testable completion of ingestion's job.

6. @finrep/db — data model & the two contracts

Prisma + Postgres. UUID PKs (`@db.Uuid`), snake_case columns via `@map`, money as `Decimal(18,2)`, snapshots/rows as `JSONB`. Prisma `binaryTargets = ["native", "debian-openssl-3.0.x"]` pins the engine binary to the container runtime.

6.1 Tenant isolation contract

`organizations` 1-< `schools` (THE TENANT) 1-< every domain table (`school_id`). Users are **global** identities; tenant access is `memberships(user_id, school_id, role)`. Every domain row carries `school_id`; `schools` carries `organization_id`.

6.2 Reproducibility contract

A `StatementSnapshot.payload` (the `ReportBundle`) is fully reproducible from: (1) immutable `imports.rows` `JSONB` (parsed `NormalizedRow[]` at upload, never mutated) + (2) `mappings.version` + (3) `standard_chart_versions.version` + (4) `engine_version`. Re-running `generateReports` on the same engine version yields **byte-identical** payload. Imports append-only; corrections create a *new* import; mappings/charts versioned, never updated in place.

6.3 Model map

```
Organization < School < FiscalPeriod < Import / StatementSnapshot
  < Membership > User (global)
  < Invitation
  < Mapping (versioned: @@unique[schoolId, version])
  < StatementSnapshot
  < Subscription (1:1)
User < RefreshToken (jti, rotation, lastActivityAt)
StandardChartVersion (global, versioned)
AuditLog (org/school/user nullable; SetNull on delete)
```

Enums: `MembershipRole` {owner, accountant, viewer}, `MembershipStatus` {active, invited}, `ImportRole` {cy, py, audit}, `SubscriptionStatus` {trialing, active, past_due, canceled, none}. Schema-only-until-later: `User` credential fields + `Subscription` exist now; their *logic* lands in 1B/1D respectively.

7. @finrep/api — NestJS backend

ESM NestJS 11. Modules: **Config, Prisma, Health, Reports, Auth, Schools, Organizations, Audit**. No global route prefix (web's Vite proxy strips `/api`).

7.1 Auth subsystem

Password hashing (`password.service.ts`) — PBKDF2-HMAC-SHA256, **600k iters**, 16-byte salt, 64-byte key; constant-time `timingSafeEqual` verify with length-guard. Matches smartbot + the db seed (change them in lockstep).

Tokens (`token.service.ts`) — access JWT (~15m) + refresh JWT (~30d). Refresh tokens are **persisted and ROTATED on use** (old `revokedAt`, new issued), with a **7-day inactivity window** and a `jti` surfaced into the access token's `sid` claim for precise multi-session activity-touch. `revokeAll` (logout/reset) and `revokeAllExcept(keepJti)` (password change keeps the current session, kills the rest).

Flows (`auth.service.ts`): - `register` → strength check, dup-email guard, create user (`emailVerified:false`), send verification email. - `verifyEmail` / `resendVerification` → token + expiry. - `login` → reject if `lockedUntil` in future; require `emailVerified`; on bad password increment `failedLoginAttempts`, lock after threshold; on success reset counters and issue token pair. - `changePassword` → verify current, re-hash, reset counters, `revokeAllExcept(currentSession)`. - `forgotPassword` / `resetPassword` → reset code + expiry, then `revokeAll`.

JWT secret resolution (`configuration.ts`) — prod **fails fast** if `JWT_SECRET` is unset, the dev default, or `< 32` chars; dev with no secret generates an *ephemeral random* per-process secret (never the well-known constant) so tokens can't be forged.

7.2 Guards & RBAC

- **JwtAuthGuard** — verifies Bearer access token, loads user → `req.user`, best-effort `touchActivity(sid)` (never blocks the request).
- **RolesGuard** — runs after `JwtAuthGuard`; resolves target school from `:schoolId` param ► `X-School-Id` header ► body; requires an **active** membership whose role ∈ `@Roles(...)`. No membership → 403 (tenant isolation); wrong role → 403.
- Decorators: `@Roles()`, `@CurrentUser()`, `@CurrentSession()`.

7.3 Audit

`AuditService.write` — shared best-effort writer for role changes, removals, invite revokes, school/org/profile/password changes. **Never logs secrets** (no passwords/hashes/tokens). A write failure is logged but never blocks the mutation it accompanies.

7.4 Endpoint catalogue

```
GET /health
POST /reports/generate [JwtAuthGuard] stateless generateReports() wrapper
Auth /auth/{register,verify-email,resend-verification,login,refresh,logout}
/auth/me [GET|PATCH] /auth/change-password
/auth/{forgot-password,reset-password}
Schools POST /schools | GET /schools
GET /schools/{:id}/members
PATCH /schools/{:id}/members/{:userId} [Roles: owner]
DELETE /schools/{:id}/members/{:userId} [Roles: owner]
PATCH /schools/{:id} [Roles: owner]
GET/POST/DELETE /schools/{:id}/invitations/{:invId}
POST /invitations/accept
Orgs GET /organizations/me | PATCH /organizations/{:orgId}
```

Note: `POST /reports/generate` is a server-side compute convenience — the web preview computes statements **client-side** via `@finrep/engine`, so this endpoint is not on the hot path.

8. @finrep/web — React 19 SPA

Vite 7 + React 19 (automatic JSX runtime) + Tailwind. Consumes engine/ingestion **from source** via Vite alias (§3.1), so report previews run fully client-side.

8.1 Routing & contexts

`App.jsx`: `react-router-dom` with `PublicOnlyRoute` (auth pages) and `ProtectedRoute` → `AuthedLayout` wrapping a single `SchoolProvider` over both Dashboard and Settings (consistent active-school/role context). `verify-email` is reachable logged-in or out (reads `?token`).

- `AuthContext` — owns the `user`; on mount rehydrates via `/auth/me` if an access token exists, with a `ready` gate so the router never flashes `/login` for a logged-in user. Listens for the `auth:logout` window event.
- `SchoolContext` — active school + role for the switcher and report preview.
- `AppContext` — intake/datasets/results state for the dashboard.

8.2 API client (`lib/api.js`)

Axios instance, base `VITE_API_URL || '/api'`. Tokens in `localStorage`. **Proactive refresh**: a request interceptor rotates the access token if it expires within a 2-min skew, deduped via a single in-flight promise (no stampede). Response interceptor does a one-shot 401 retry through `/auth/refresh`. On hard refresh failure → clear tokens, dispatch `auth:logout` (`AuthContext` owns the redirect, keeping the client router-agnostic). JWT `exp` decoded inline (no lib).

8.3 Reports & export

`components/reports/` renders SOA / SFP / SCF / Net Assets with shared `cells.jsx` primitives, plus zoom/pan + expand overlays and a report picker. **Excel export** (`lib/excel.js`, `ExcelJS`) is **dynamically imported** only on export to keep the initial bundle lean. Build splits manual chunks: `react-vendor`, `framer`, `xlsx`.

9. End-to-end flows

9.1 Report generation (web preview — the hot path)

```
Upload .xlsx/.csv
@finrep/ingestion: adapter → NormalizedRow[] + SheetMetadata
  ↳ classifyRole/resolveRoles → cy/py/audit slots (+ conflicts)
  ↳ inferPeriod → date/type
  ↳ user sets period + period-end + begin balances
@finrep/engine generateReports({cyData, pyData, auditData, school})
  ↳ ReportBundle {soaResults, sfpResults, scf, netAssets, unmapped, validation, meta, lineage}
  ↳ render tabs / Print PDF / lazy ExcelJS export
```

9.2 Auth (login + authed request)

```
POST /auth/login {access(~15m), refresh(~30d persisted+jti)}
  web stores both in localStorage
authed request:
  interceptor attaches Bearer; if near-expiry /auth/refresh (rotate) first
  JwtAuthGuard verify req.user, touchActivity(sid)
  RolesGuard resolves school + checks active membership role
hard refresh fail auth:logout event AuthContext clears user ProtectedRoute /login
```

10. Security model (summary)

Control	Implementation
Password storage	PBKDF2-SHA256, 600k iters, per-user salt, constant-time verify
Brute force	failedLoginAttempts + lockedUntil logout
Email trust	emailVerified required to log in; tokenized verify + resend
Session	Access/refresh JWT, refresh rotation , 7-day inactivity, revoke-all / revoke-all-except
JWT secret	Prod fails fast on weak/default/short; dev ephemeral random
Tenant isolation	Global users + memberships ; RolesGuard enforces active membership + role
Audit	Best-effort AuditLog , secrets never recorded
Token transport	Bearer header; web client proactive-refresh + one-shot 401 retry
Immutability	Append-only imports; versioned mappings/charts; reproducible snapshots

11. Infrastructure & deployment

docker-compose.yml (local dev): postgres:16 (host 5434→5432, healthcheck, named volume finrep_pgdata) + api (multi-stage Dockerfile, env_file: .env , talks to postgres:5432 internally, host port API_HOST_PORT:-8000 , /health healthcheck). Optional web behind the web profile (nginx on 8080).

API image: turbo prune @finrep/api --docker (prunes web out), multi-stage on node:22-slim . prisma generate at build time; prisma migrate deploy at start time (idempotent, via start.sh). Pinned Prisma engine target + dev-dep pruning + *.map strip (~716 MB → ~658 MB).

Config (configuration.ts): required DATABASE_URL ; PORT=8000 , WEB_ORIGIN ; jwt. {secret,accessTtl=900s,refreshTtl=30d} ; smtp.* ; stripe.* (read but **unused until 1D**).

Dev (vite.config.ts): server 0.0.0.0:5173 , /api proxied to :8000 with prefix strip, *.trycloudflare.com tunnel hosts allowed, fs.allow repo root (for source-consumed packages).

12. Build, dev & test workflow

```
# Dev (web only; preview computes client-side)
pnpm dev                                # turbo run dev --filter=@finrep/web → :5173

# Full stack
corepack enable pnpm && pnpm install
pnpm -F @finrep/db generate             # Prisma client
docker compose up -d --build            # postgres + api on :8000

# Build (turbo ^build ordering: db + engine → api)
pnpm build | pnpm typecheck | pnpm lint

# Engine tests + regression harness
pnpm --filter @finrep/engine test       # vitest: regression, purity, validation, openingNA
API_URL=http://localhost:8000 node scripts/verify-engine.mjs
```

`turbo.json`: `build.dependsOn ["^build"]` (outputs `dist/**`, `.vite/**`); `dev` persistent + uncached; `test` / `typecheck` also `^build`.

13. Phase → component map

Phase	Status	Touches	What lands
1A Foundation	✓	api (Config, Prisma, Health, Reports)	service boots, <code>/health</code> , stateless <code>/reports/generate</code>
1B Auth & access	✓	api (Auth, Schools, Orgs, Audit, guards), db (User/Membership/Invitation/RefreshToken logic), web (auth pages, contexts, settings)	accounts, JWT+rotation, RBAC, invitations, settings, audit
1C Save & history	👉 soon	db models already defined (Import/Mapping/StandardChartVersion/StatementSnapshot) → activate write/read paths	persisted immutable imports, versioned mappings/charts, reproducible snapshots, history UI
1D Billing	📋	db <code>Subscription</code> exists; <code>stripe.*</code> config reserved	Stripe customers/subs, trials, plan gating, webhooks

The schema for 1C/1D already exists; those phases wire **logic** onto an existing data model — the reproducibility and tenancy contracts were designed up front so later phases don't require migrations-of-record.

14. Constraints, sharp edges & future considerations

- **Verbatim legacy port.** SOA/SFP/SCF preserve quirky behaviors (explicit tuition acct list, acct-120/200 description reclass, ancillary-excluded-from-totals) for byte-identical parity. Refactors must keep the regression suite green — the *numbers* are the contract, not the structure.
- **DEFAULT_MAPPING is a single global chart (`scoa-v1`).** Per-school mapping overrides live in the `Mapping` model but aren't wired into the engine call path yet (1C work).

- **Tokens in `localStorage`**. Chosen for the SPA + proactive-refresh design; revisit vs `httpOnly` cookies if XSS surface grows.
- **`/reports/generate` is off the hot path**. Preview is client-side; the endpoint exists for server-side compute (and future snapshot persistence in 1C).
- **Build asymmetry (source vs dist)** is deliberate (\$3.1) — don't unify it.
- **SCF requires a beginning dataset** (`audit` preferred, else `py`); without one it returns `null` by design.
- **Opening-NA `plug` case needs confirmation** — it absorbs *any* TB imbalance, not just the omitted equity row.

15. Appendix — quick reference

Engine public API (`@finrep/engine`): `generateReports` , `calcSOA/SFP/SCF/NetAssets` , `deriveOpeningNetAssets` , `validateDataset/findUnmapped/hasEquityRow` , `SCoA` (`SCOA_CATEGORIES` , `DEFAULT_CHART/MAPPING` , `categoryOf` , `sumByAccts/Category`), `SCHOOLS/SCHOOL_OPTIONS` , `ENGINE_VERSION/MAPPING_VERSION/STANDARD_CHART_VERSION` .

Ingestion public API (`@finrep/ingestion`): `ingest/getAdapter/adapters` , `excelAdapter/csvAdapter` (+ `parseTrialBalance[Csv]`), `extractSheetMetadata/detectFiscalYear/detectExplicitDate/detectAuditStatus` , `classifyRole/resolveRoles/inferPeriod/isFiscalYearEnd` .

Key env vars: `DATABASE_URL` (req), `PORT` , `WEB_ORIGIN` , `JWT_SECRET` (req in prod, ≥ 32), `JWT_ACCESS_TTL` , `JWT_REFRESH_TTL` , `SMTP_{HOST,PORT,USER,PASS,FROM}` , `STRIPE_SECRET_KEY` / `STRIPE_WEBHOOK_SECRET` (1D), `API_HOST_PORT` , `VITE_API_URL` .

Versions: engine `0.1.0` · mapping `map-v1` · chart `scoa-v1` .

Generated from a direct read of the codebase, June 2026. Reflects committed code plus the in-progress 1B working tree.